CMSC 420: AVL Trees

Justin Wyss-Gallifent

January 16, 2026

1 Problems **2**

1 Problems

1. Consider the following AVL tree:

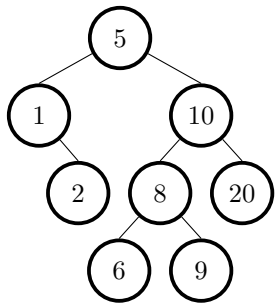

Figure 1: An AVL Tree

The following questions are non-cumulative:

(a) Suppose we insert a key x and the tree doesn't need rebalancing. Assuming x is an integer and all the keys are different, what can we say about x? You may use specific values or inequalities to express your answer.

What is true about x?	$x < 1$ or $x > 10$

(b) Suppose we delete a key x, using inorder predecessors when replacements are needed, and the tree doesn't need rebalancing. What can we say about x? You may use specific values or inequalities to express your answer.

What is true about x?	$x = 6, 8, 9, 10$

(c) Suppose we delete the key $x = 2$. How many single rotations will be necessary to rebalance the tree? Note that a left-right rotation would count as two "single rotations".

How many single rotations?	2

2. Here is a diagram of the right rotation of a node:

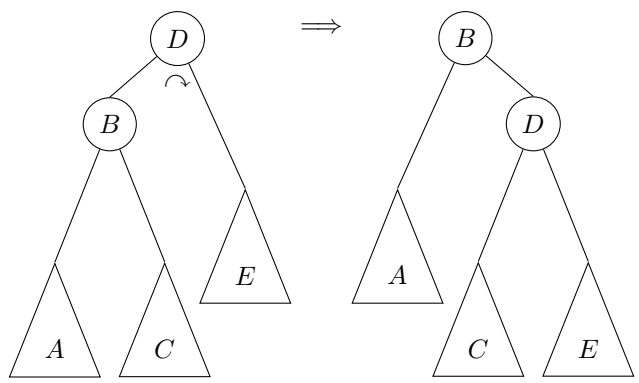

Figure 2: Right Rotation at a Node

Suppose that nodes B and D each contain values `B.leftheight`, `B.rightheight`, `D.leftheight`, and `D.rightheight` which contain the height of their respective subtrees?

(a) After the rotation what would `B.leftheight` be in terms of the values before the rotation?

Solution:
Remains the same.

(b) After the rotation what would `B.rightheight` be in terms of the values before the rotation?

Solution:
`B.rightheight = 1 + max(B.rightheight,D.rightheight)`

(c) After the rotation what would `D.leftheight` be in terms of the values before the rotation?

Solution:
`D.leftheight = B.rightheight`

(d) After the rotation what would `D.rightheight` be in terms of the values before the rotation?

Solution:
Remains the same.

3. In class we drew pictures of the effect of a right rotation on a left-left heavy AVL tree after insertion and the effect of a left-right rotation on a left-right heavy AVL tree after insertion. Draw appropriate pictures of the effect of a left rotation on a right-right heavy AVL tree after insertion and the effect of a right-left rotation on a right-left heavy AVL tree after insertion.

   Solution:
   These should essentially just be mirror images of those.

4. Starting with the following AVL tree:

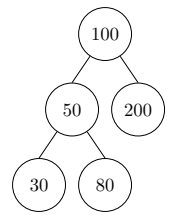

Figure 3: An AVL Tree

Perform the following operations cumulatively in the order given. For each operation draw:

- The tree after the operation but before any required rebalancing.
- The tree after each step of any required rebalancing operations. For example if a single rotation is required, just draw the result, but if a double-rotation is required draw the result after each component rotation.

(a) Insert 90.
Solution:
This requires a left rotation at 50 followed by a right rotation at 100:

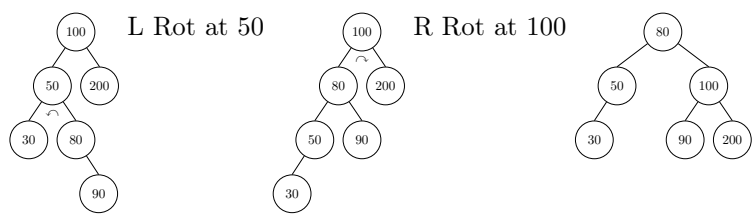

Figure 4: Insert 90

(b) Insert 10.
Solution:
This requires a right rotation at 50:

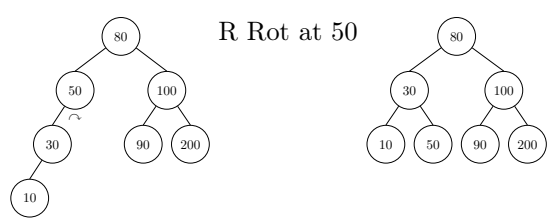

Figure 5: Delete 10

5. Given the following AVL tree:

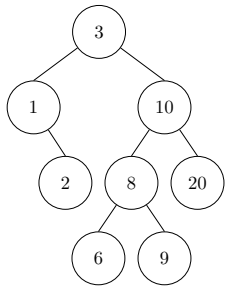

Figure 6: An AVL Tree

For the following questions start with the above tree. In other words the questions do not build from one another!

(a) Show the process (each step) through which the key 5 is inserted into the tree.

Solution:
We have:

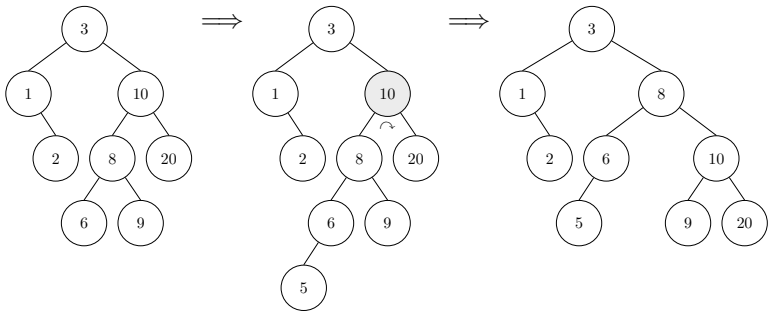

Figure 7: Inserting 5

(b) Show the process (each step) through which the key 2 is deleted from the tree.

Solution:

We have:

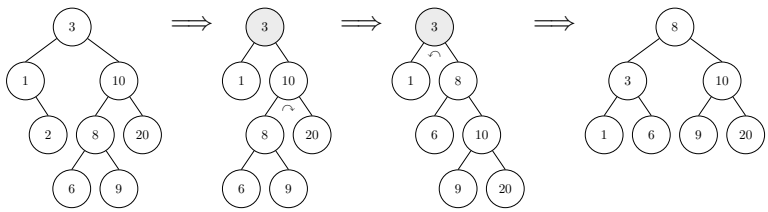

Figure 8: Deleting 2

6. In class we showed that for an AVL tree with height h and n nodes that when h is even then $N(h) > 2N(h-2)$ implies that $h = \mathcal{O}(\lg n)$, where $N(h)$ is the minimum number of nodes in an AVL tree of height h. Prove that the statement is also true when h is odd.

Solution:
Noting that $N(1) = 2$ we have:

$$N(h) > 2N(h-2) > 2^2 N(h-4) > ... > 2^{\lfloor h/2 \rfloor} N(1) > 2^{h/2-1}(2) = 2^{h/2}$$

Taking the lg of both sides yields:

$$\lg N(h) > \frac{h}{2}$$

And so:

$$h < 2 \lg N(h)$$

Since $N(h)$ is the minimum number of nodes in an AVL tree of height h then for any AVL tree of height h if there are n nodes then $N(h) \leq n$ and so:

$$h < 2 \lg n$$

The result follows.

7. Here is the code (in Python) for the left rotation in an AVL tree:

```
def rotateleft(root: Node) -> Node:
    RightChild = root.rightchild
    RightLeftSubtree = RightChild.leftchild
    RightChild.leftchild = root
    root.rightchild = RightLeftSubtree
return RightChild
```

Suppose that the `Node` class also contains variables `Node.leftheight` and `Node.rightheight` which store the heights of the left and right subtrees.

Write down the two (and only two!) necessary lines which will go immediately before the `return` statement so that these two variables are updated wherever necessary. Your result does not need to be proper Python, pseudocode will suffice.

Solution:

```
def rotateLeft(root: Node) -> Node:
    RightChild = root.rightchild
    RightLeftSubtree = RightChild.leftchild
    RightChild.leftchild = root
    root.rightchild = RightleftSubtree
    # The next two lines.
    root.rightheight = RightChild.leftheight
    RightChild.leftheight
      = 1 + max(root.leftheight,RightChild.leftheight)
    #
    return RightChild
```

8. Suppose b_h equals the number of different AVL tree structures of height h. We don't care about the contents, just the overall structure.

   (a) Calculate b_0 and b_1 via illustrations.

   Solution:
   Easy - just draw the trees!
   Here is b_0:

Figure 9: AVL Structures of Height 0

   Here is b_1:

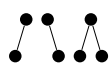

Figure 10: AVL Structures of Height 1

   We get $b_0 = 1$ and $b_1 = 3$.

   (b) Prove that for $h \geq 2$ we have:

$$b_h = 2b_{h-1}b_{h-2} + b_{h-1}^2$$

   Solution:
   An AVL tree of height h is comprised of a root node and two subtrees each of which are also AVL trees. These sub(AVL)trees must differ in height by at most 1 so that the tree itself is an AVL tree and the maximum height of these subtrees must be $h - 1$.
   The only way to do this is to have either:

   - A left sub(AVL)tree of height $h - 1$ and a right sub(AVL)tree of height $h - 2$. There are $b_{h-1}b_{h-2}$ such combinations.
   - A left sub(AVL)tree of height $h - 2$ and a right sub(AVL)tree of height $h - 1$. There are $b_{h-2}b_{h-1}$ such combinations.
   - Both sub(AVL)trees are of height $h-1$. There are $b_{h-1}b_{h-1}$ such combinations.

   The sum of these is exactly the expression given.

   (c) Use this to calculate b_h for $h = 2, 3, 4$.

   Solution:
   We have:

$$b_2 = 2b_1b_0 + b_1^2 = 2(3)(1) + (3)^2 = 15$$
$$b_3 = 2b_2b_1 + b_2^2 = 2(15)(3) + (15)^2 = 315$$
$$b_3 = 2b_3b_2 + b_3^2 = 2(315)(15) + (315)^2 = 108675$$

(d) Verify your calculation for $h = 2$ by illustrating all possible structures.

Solution:

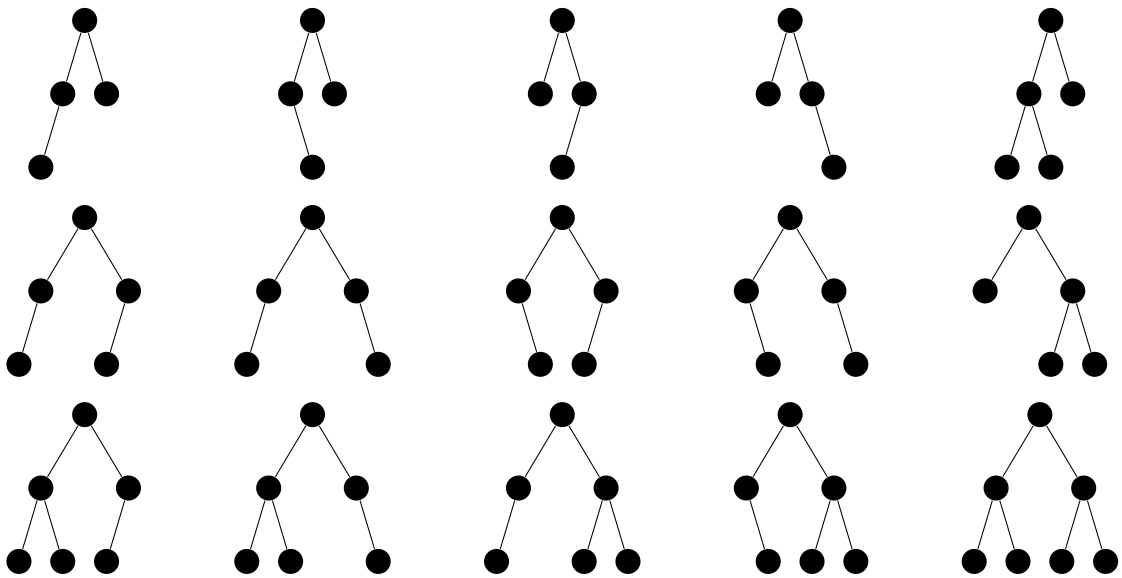

Figure 11: All AVL Structures of Height 2

9. The following statement is either true or false. State which you believe it to be and justify. Note: The depth of a node is the distance (number of edges) from the node to the root.

   Statement: For any AVL tree if L_i and L_j are leaves, then:

$$\text{depth}(L_i) - \text{depth}(L_j) \in \{-1, 0, 1\}$$

   Solution:
   False. For example:

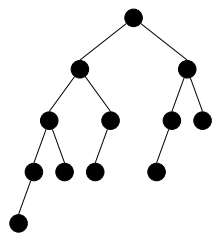

Figure 12: Example Of Leaf Node Depth Difference of 2

10. Define an AVL-2 tree to be a BST where each node v satisfies balance$(v) \in \{-2, -1, 0, 1, 2\}$. Let $N(h)$ be the minimum number of nodes in an AVL-2 tree of height h. Prove that $h = \mathcal{O}(\lg n)$ for the case where h is a multiple of 3.

Solution:
If $N(h)$ is the minimum number then we know two things:

- An AVL-2 tree of height h will have one subtree of height $h-1$ and the other of height $h-3$.
- We have $N(h-1) > N(h-3)$.

From here we can say:

$$N(h) = 1 + N(h-1) + N(h-3) > 2N(h-3) + 1 > 2N(h-3)$$

Thus we have:

$$N(h) > 2N(h-3) > 2^2 N(h-6) > ... > 2^{h/3} N(0) = 2^{h/3}(1) = 2^{h/3}$$

Since in general $n > N(h)$ we have:

$$\begin{aligned} n &> 2^{h/3} \\ \frac{h}{3} &< \lg n \\ h &< 3 \lg n \end{aligned}$$

Thus $h = \mathcal{O}(\lg n)$.

11. Suppose an AVL tree with height h has n nodes. Let F_i be the ith Fibonacci number, so $F_0 = 0$, $F_1 = 1$, and $F_j = F_{j-1} + F_{j-2}$ for $j \geq 2$. Prove that:

$$n \geq F_{h+3} - 1$$

Solution:
Let $N(h)$ be the minimum possible nodes in an AVL tree with height h. we prove by strong induction that for all $h \geq 0$ that we have $N(h) = F_{h+3} - 1$.
We have $N(0) = 1$ and $F_{0+3} = F_3 = 2$ so $N(0) = F_{0+3} - 1$.
We have $N(1) = 2$ and $F_{1+3} = F_4 = 3$ so $N(1) = F_{1+3} - 1$.
An AVL tree of height h with the minimum number of nodes will have one subtree with height $h-1$ and the other with height $h-2$. Thus:

$$N(h) = 1 + N(h-1) + N(h-2)$$

Then for $j \geq 2$ we have:

$$\begin{aligned} N(h) &= 1 + N(h-1) + N(h-2) \\ &= 1 + F_{h-1+3} + F_{h-2+3} \\ &= 1 + F_{h+2} - 1 + F_{h+1} - 1 \\ &= F_{h+3} - 1 \end{aligned}$$

Since $n \geq N(h)$ we have our conclusion.

12. Prove that an AVL tree with n nodes has height $\Omega(\lg n)$.

Solution:
For an AVL tree with height h the maximum number of nodes occurs when the tree is perfect, meaning it has $2^{h+1} - 1$ nodes.

For any AVL tree with height h it therefore follows that:

$$n \leq 2^{h+1} - 1$$

So then:

$$\begin{aligned} 2^{h+1} - 1 &\geq n \\ 2^{h+1} &\geq n + 1 \\ h &\geq \lg(n+1) - 1 \end{aligned}$$

The result follows.

13. Prove by induction that for any integer $\delta \geq 1$ it's possible to construct an AVL tree containing two leaves whose depth differs by δ.

Solution:
We will actually prove that it can be done with one of the leaves being as deep as possible in the tree.

Base Case: For $\delta = 1$ we have the following.

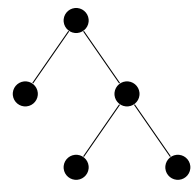

Figure 13: Base Case

Inductive Step: Assume for any $\delta \geq 1$ we can construct an AVL tree T such that L_2 is as deep as possible and is δ deeper than L_1.
Construct a new tree with a new root where one child is T and the other child is a perfect tree of height $1 + \text{height}(T)$.
Then for any leaf L_3 in the perfect tree:

$$\text{depth}(L_3) = 1+\text{height}(T) = 1+(\text{depth}(L_2) = 1+\text{depth}(L_1)+\delta = \text{depth}(L_1)+(\delta+1)$$

Thus L_1 and L_3 differ by $\delta + 1$. Note also that L_3 is as deep as possible.

14. For some fixed but unknown positive integer k define an AVL-k tree to be a BST where each node v satisfies:

$$\text{balance}(v) \in \{-k, ..., -2, -1, 0, 1, 2, ..., k\}$$

Let $N(h)$ be the minimum number of nodes possible in an AVL-k tree of height h. Use this to prove that $h = \mathcal{O}(\lg n)$. For simplicity of some calculations you can assume that $h \geq k$.

Solution:
If $N(h)$ is the minimum number then we know two things:

- Such an AVL-k tree of height h will have one subtree of height $h-1$ and the other of height $h-(k+1)$.
- We have $N(h-1) > N(h-(k+1))$.

From here we can say:

$$N(h) = 1+N(h-1)+N(h-(k+1)) > 2N(h-(k+1))+1 > 2N(h-(k+1))$$

Thus we have:

$$\begin{aligned} N(h) &> 2N(h-(k+1)) \\ &> 2^2 N(h-2(k+1)) \\ &> ... \\ &> 2^{\lfloor h/(k+1) \rfloor} N(h \mod k+1) \end{aligned}$$

Since in general $n \geq N(h)$ and $\lfloor h/(k+1) \rfloor > h/(k+1) - 1$ we have:

$$n > 2^{h/(k+1)-1} N(h \mod k+1)$$

From here:

$$\begin{aligned} 2^{h/(k+1)-1} N(h \mod k+1) &< n \\ 2^{h/(k+1)-1} &< \frac{n}{N(h \mod k+1)} \\ \frac{h}{k+1} - 1 &< \lg\left(\frac{n}{N(h \mod k+1)}\right) \\ \frac{h}{k+1} &< \lg\left(\frac{n}{N(h \mod k+1)}\right) + 1 \\ h &< (k+1)\lg\left(\frac{n}{N(h \mod k+1)}\right) + k + 1 \end{aligned}$$

For any given h the value of $N(h \mod k+1)$ is some fixed constant and so $h = \mathcal{O}(\lg n)$.

15. Consider the following AVL tree:

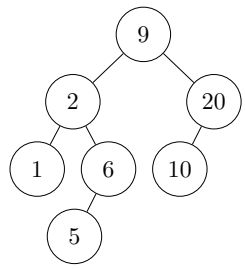

Figure 14: An AVL Tree

The following questions are non-cumulative. For each question assume x is a positive integer and all the keys are different.

Please read these questions carefully!

(a) List all the keys x with balance$(x) = \pm 1$. List in increasing order.

Possible Keys	$x = 2, 6, 9, 20$

(b) Suppose we insert a key x and the tree does not need rebalancing. What are the possible values of x? You may use specific values or inequalities to express your answer.

Possible Keys	$x = 7, 8$ or $x > 20$

(c) Suppose we delete a key x, using inorder predecessors when replacements are needed, and the tree does need rebalancing. What are the possible values of x? List in increasing order.

Possible Keys	$x = 1, 2, 10, 20$

(d) Suppose we delete a key x, using inorder successors when replacements are needed, and the tree does need rebalancing. What are the possible values of x? List in increasing order.

Possible Keys	$x = 1, 9, 10, 20$

(e) Suppose we delete the key $x = 20$. How many single rotations will be necessary to rebalance the tree? Note that a LR or RL rotation would count as two "single rotations". [5 pts]

How Many?	2

16. If a binary tree has the structure shown:

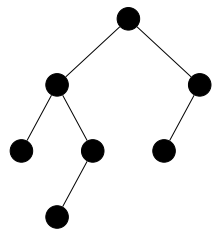

Figure 15: A Binary Tree

If we perform a right rotation at the root, draw the resulting tree structure.

Solution:

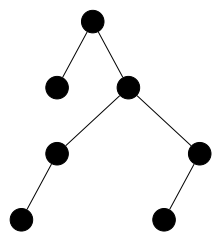

Figure 16: After a Right Rotation

17. Suppose `x` is the root of a nonempty binary tree. Suppose we run this algorithm:

```
x = root
while x != None:
    while x.left != None:
        rightrotation at x
        x = x.parent
    end while
    x = x.right
end while
```

What will the binary tree look like after this is run? A single sentence will suffice.

Solution:
It will be a linked list going down and to the right.

18. Here is a diagram of the right rotation of a node:

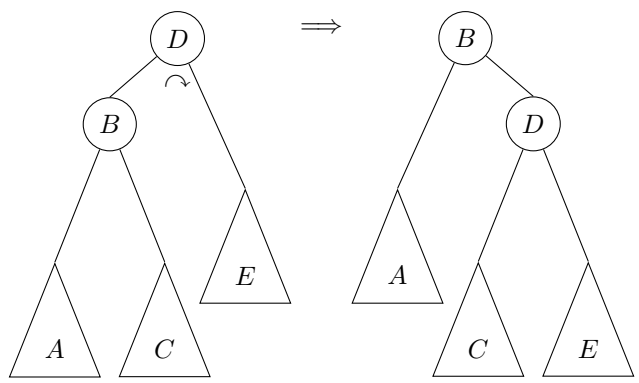

Figure 17: Right Rotation at a Node

Suppose we have the values `B.leftheight`, `B.rightheight`, `D.leftheight`, and `D.rightheight` which store the heights of the subtrees. Out of these four only `D.leftheight` and `B.rightheight` change with the rotation. What would the new values be in terms of the old values?

`D.leftheight =`	`B.rightheight`
`B.rightheight =`	`1+max(B.rightheight,D.rightheight)`

19. Define $N(h)$ to be the minimum number of nodes possible in an AVL tree with height h.
We know from class that $N(h) > 2N(h-2)$.
Use digging down to prove that $h = \mathcal{O}(\lg n)$ when h is odd.

Proof:
We have:

$$\begin{aligned} N(h) &> 2N(h-2) \\ &> 2^2 N(h-2) \\ &> ... \\ &> 2^{(h-1)/2} N(1) \end{aligned}$$

But $N(1) = 2$ and $n \geq N(h)$ and so:

$$\begin{aligned} n \geq N(h) &> 2 \cdot 2^{(h-1)/2} \\ 2^{(h-1)/2} &< n/2 \\ (h-1)/2 &< \lg(n/2) \\ h &< 2\lg(n/2) + 1 \end{aligned}$$

The result follows.

Note: This can also be done with a floor, yielding $N(h) > 2^{\lfloor h/2 \rfloor} N(1)$ and going from there.